

Vitto Lending Decision System — Architecture

Write-up

Overview

This project is a lightweight lending decision system for MSME loan applications. It collects a business profile and loan request, evaluates the request using an explainable scoring model, and returns a structured decision with a credit score and reason codes.

Stack: React (SPA) · Node.js/Express (REST API) · PostgreSQL (system of record) · MongoDB (audit trail)

High-level Architecture

```

React SPA
-> Node/Express API
  -> Validation middleware (transport validation)
  -> Use cases (application orchestration)
    -> Decision engine (pure business logic)
    -> Repositories (Postgres persistence)
    -> Audit logger (Mongo persistence)
  
```

Key design choice: keep the decision logic isolated so it is testable and easy to explain.

Data Storage Strategy (Postgres + Mongo)

PostgreSQL is the system of record for: - Business profiles - Loan applications - Credit decisions (including reason codes and derived metrics snapshot)

MongoDB is used for: - An append-only audit trail of events such as profile creation, application creation, and decision creation

Why split it this way: - The core domain data is relational and benefits from constraints and joins - Audit logs are naturally document-like and append-only - MongoDB gets a clear, defensible responsibility without forcing dual-ownership of core state

Audit failure behavior: - If Postgres commit succeeds but Mongo audit write fails, the API still returns success (the core decision is already persisted); the server logs the audit failure for visibility.

Decision Engine Design

The decision engine is implemented as a pure domain module:

Hard boundaries (non-responsibilities): - does not know about Express - does not know about SQL - does not write logs directly - does not fetch data itself

It accepts already-loaded inputs (business profile + loan application) and returns: -

`APPROVED` or `REJECTED` - a computed credit score (0–100) - one or more reason codes - derived metrics used to explain the decision

Why this design: - Deterministic, unit-testable logic without needing a running server or database - Prevents business logic from being scattered inside HTTP handlers or persistence code - Makes it easier to defend thresholds and outcomes in the README and interview discussion

Scoring Model (Explainable and Documented)

The scoring model prioritizes explainability over complexity. Signals include: - Revenue-to-EMI ratio (monthly revenue vs estimated monthly repayment) - Loan-to-revenue multiple - Tenure risk (very short or very long tenures) - Basic consistency checks (loan amount disproportionate to revenue)

The model: - starts with a base score of 100 - applies penalties per risk signal - can hard-reject on invalid or extreme inconsistencies

Reason codes: - `LOW_REPAYMENT_CAPACITY` - `HIGH_LOAN_RATIO` - `EXTREME_TENURE` - `DATA_INCONSISTENCY` - `INVALID_PAN_FORMAT`

All thresholds and assumptions are documented in the README.

Edge Case Handling

Edge case handling is implemented at both the frontend and backend boundaries: -

Missing fields: client-side required field checks and server-side validation - Invalid formats: structured validation errors (e.g., malformed PAN, negative numbers) - Conflicting data: valid request is accepted but decision is rejected with explainable reason codes

Goal: no unhandled exceptions in normal flows; error responses are consistent and structured.

Tradeoffs Chosen

- **Synchronous decisioning:** chosen for simplicity and reliability in a 1-day sprint; async job queue was intentionally deferred (bonus only).
- **Format-only PAN validation:** chosen because real verification would require external integrations outside assignment scope.
- **Minimal UI:** prioritized clarity and correctness over complex UI states.
- **Paise storage in Postgres:** avoids floating-point rounding issues by storing money as integers.

Improvements With More Time

- Async decision processing (job + polling endpoint)
- More robust audit/event taxonomy and dashboards
- Decision engine calibration using sample datasets and more nuanced risk weights
- Stronger observability: structured logs, metrics, tracing
- Stronger API hardening: stricter rate limiting, abuse detection, request size constraints
- Admin dashboard to view audit trail and decision trends